

SESSION - 17

Services, Components, Servlets



Table of Contents

Table of Contents **1**

OSGi Services, Components and Sling Servlets ***2***

Introduction **2**

Objectives **2**

Understanding OSGi concepts ***2***

Software Modularity **2**

What is OSGi? **3**

What is a Service ***3***

What is a Component ***4***

Exercise 1: Write your first service 4

Exercise 2: Understand various annotations. 7

What is a Servlert ***7***

Sling Servlets **7**

Servlet Registration **8**

Types of Servlets **9**

Exercise 3: Create a resourceType based get servlet 9

Exercise 4: Create a path based get servlet 9

Exercise 5: Create a post servlet : 10

OSGi Services, Components and Sling Servlets

Introduction

OSGi services and components are the core of AEM project, without these we can't work on a actual project implementation. Similarly use of servlet is also very crucial for dynamic projects. In this chapter we are going to cover these topics in detail with example exercises.

In previous session we have completed:

- installed Eclipse in our system.
- We have a working project based on archetype 23.

Objectives

After completing this module, you will be able to:

- Understand and create OSGi Services
- Understand and create OSGi Components
- Understand and create Sling Servlets
- Generate code bundle from Eclipse

Note: We will learn all above points using exercises.

Understanding OSGi concepts

Software Modularity

In modern times, complex software can be thought of as a collection of various modules or components. These modules are normally independent of each other and modification in one module does not affect the other modules.

These modules interact with each other via an API. The API is defined as a set of classes and methods which can be used from other components.

If a module uses an API from another module, it has a dependency on the other module, i.e., it requires the other module exists and works correctly.

A module which is used by other components should try to keep its API stable. This avoids that a change affects other modules. But it should be free to change its internal implementation.

What is OSGi?

OSGi stands for ***Open Service Gateway initiative.*** It is a Java framework for developing and deploying modular software programs and libraries.

OSGi has two parts.

- The first part is a specification for modular components called bundles, which are commonly referred to as plug-ins. The specification defines an infrastructure for a bundle's life cycle and determines how bundles will interact.
- The second part of OSGi is a Java Virtual Machine (JVM)-level service registry that bundles can use to publish, discover and bind to services in a service-oriented architecture (SOA).

The components and services can be dynamically installed, activated, de-activated, updated and uninstalled.

The OSGi specification has several implementations, for example, **Equinox**, **Knopflerfish**, and **Apache Felix**. AEM uses Apache Felix implementation. A bundle is the smallest unit of the modularization which means in OSGi, a software component is a bundle.

In AEM, along with out of the box bundles, we can also install our custom bundles very easily.

As an AEM developer, it is important that you are familiar with these Sling annotations:

- **@Component**
- **@Service**
- **@Reference**
- **@Property**

For more information about these annotations, see SCR Annotations.

Note: It is also important that you understand the difference between components and services.

What is a Service

The classical view of a service is nothing, but work done for another part of AEM. So is it similar to a method call? The answer is no. How does a service differ from a method call?

A service implies a contract between the provider of the service and its consumers. Consumers typically are not worried about the exact implementation behind a service (or even who provides it) as long as it

follows the agreed contract, suggesting that services are to some extent substitutable. Using a service also involves a form of discovery or negotiation, implying that each service has a set of identifying features.

A service is an object that is registered in the OSGi Service Registry and can be looked up using its interface name(s). The only prerequisite is that a service should implement some interface. For example, I could register a runnable object under the java.lang.Runnable interface and clients could look it up using that interface name.

What is a Component

An AEM back-end component refers to managing the life cycle of a class in an OSGi environment. All OSGi services should be a component so that AEM handles the life cycle of a service. For example, the AEM handles when the component is activated, stopped, and so on.

A "component" tends to be an object whose lifecycle is managed, usually by a component framework such as Declarative Services (DS), Blueprint or iPOJO.

A component may have any of the following features, in combination or alone:

1. A component may be started and stopped; this would be considered an "active" component, though that is also an informal term. A component that doesn't need to be started or stopped is called passive.
2. A component may publish itself as an OSGi service.
3. A component may bind to or consume OSGi services.
4. In general, using a component framework is the easiest way to work with OSGi services because the framework will manage the binding to the services that you want to consume. For example, you could say that your component "depends on" a particular service, in which case the component will only be created and activated when that service is available -- and also it will be destroyed when the service becomes unavailable.

Exercise 1: Write your first service

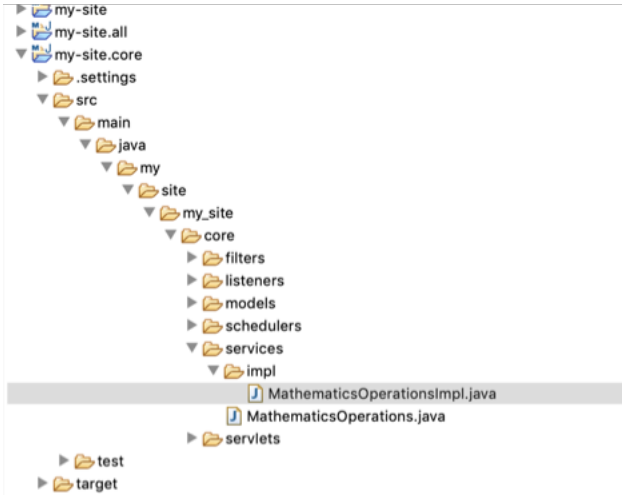
To create your own service follow below steps:

Go to mysite core module and We will be working on projects created using maven. Hence we will learn how to import a maven project into eclipse and then will work on that project.

To import a maven project follow below steps:

1. Go to mysite core module.
2. Create a new package/folder under package **my.site.my_site.core** with the name **services**.

3. Create a new folder under services folder with the name **impl**. Your folder structure should look like as shown:



4. Now create a java interface with the name MathematicsOperations.java inside **services** folder. You can copy the code from the exercise folder.

```
package my.site.my_site.core.services;

public interface MathematicsOperations {
    
    public int additionsOperation(int a, int b);
    
    public int multiplicationOperation(int a, int b);

}
```

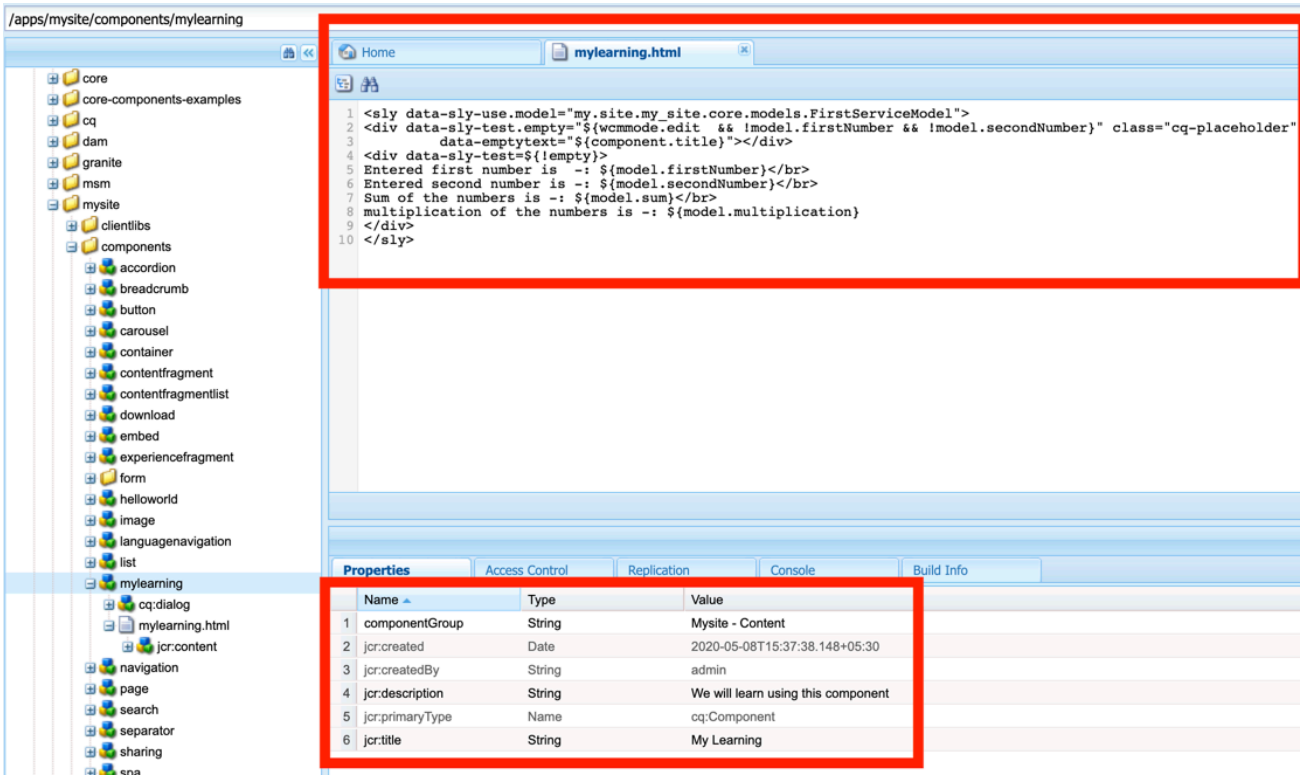
5. Also create a java class inside **impl** folder which implements the previous interfaces. You can copy the code from the exercise folder.

```
1  package my.site.my_site.core.services.impl;
2
3  import org.osgi.service.component.annotations.Component;
4
5  import my.site.my_site.core.services.MathematicsOperations;
6
7  @Component(service = MathematicsOperations.class, immediate = true)
8  public class MathematicsOperationsImpl implements MathematicsOperations {
9
10     @Override
11     public int additionsOperation(int a, int b) {
12         return a+b;
13     }
14
15     @Override
16     public int multiplicationOperation(int a, int b) {
17         return a*b;
18     }
19
20 }
21
```

6. Now your service is ready which is having two service methods **sum** and **multiplication**. These method does the sum and multiplication of two integer numbers which are provided as the input.

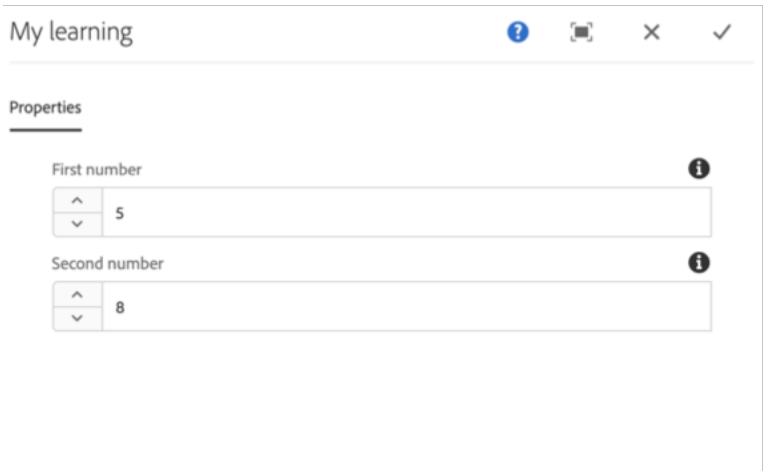
Note: This was done to showcase you how services can be written and how they work. Your actual service may have a complex business logic.

7. Now create a folder mylearning as shown:



8. We are not going to spend time here on how to create a component. You can find the code for this component in exercise folder.

9. Now create a dialog for this component which can take 2 numbers as input as shown:



10. Now Create a sling model **FirstServiceModel.java** which reads the dialog field values of this component and also calls the service methods of the service which we created. Code is given in the exercise folder.

11. Use this sling model into your components default script via including it as shown:

```
<sly data-sly-use.model="my.site.my_site.core.models.FirstServiceModel">
<div data-sly-test.empty="${wcmmode.edit && !model.firstNumber && !model.secondNumber}" class="cq-placeholder"
    data-emptytext="${component.title}"></div>
<div data-sly-test=${!empty}>
Entered first number is -: ${model.firstNumber}</br>
Entered second number is -: ${model.secondNumber}</br>
Sum of the numbers is -: ${model.sum}</br>
multiplication of the numbers is -: ${model.multiplication}
</div>
</sly>
```

12. Now build your code and deploy it on server.
13. Drag and drop this new component on page and see how the values entered from dialog is captured via sling model. Also notice you are seeing output with the business logic implemented on the page.

Now your component is ready, from which you will take two numbers as input and your sling model will call the OSGi service which you created and will return the response as per the business logic into its service methods.

In actual projects this is how we write the business logic. In this way there should not be any business logic inside your Sling model. All the business logic should go to service methods. This is an Ideal flow.

Exercise 2: Understand various annotations.

This is a self-exercise which you should finish after the session. Go through the documentation to understand various annotations which we used in the previous exercise.

What is a Servlert

A Servlet is a class used to extend the capabilities of servers that host applications accessed by means of a request-response programming model. For such applications, Servlet technology defines HTTP-specific servlet classes.

All servlets must implement the Servlet interface, which defines life-cycle methods. When implementing a generic service, we can use or extend the **GenericServlet** class provided with the Java Servlet API. The **HttpServlet** class provides methods, such as **doGet()** and **doPost(),** for handling HTTP-specific services.

Sling Servlets

In Sling, servlets can be registered as services. Whenever you create an AEM Multimodule Project using maven archetype, the default template for a Sling Servlet is generated which is located inside **core module's servlet package** as follows -

```
2  * Copyright 2015 Adobe Systems Incorporated
16 package my.site.my_site.core.servlets;
17
18 import com.day.cq.commons.jcr.JcrConstants;
32
33 /**
34  * Servlet that writes some sample content into the response. It is mounted for
35  * all resources of a specific Sling resource type. The
36  * {@link SlingSafeMethodsServlet} shall be used for HTTP methods that are
37  * idempotent. For write operations use the {@link SlingAllMethodsServlet}.
38  */
39 @Component(service = { Servlet.class })
40 @SlingServletResourceTypes(
41         resourceTypes="mysite/components/page",
42         methods=HttpConstants.METHOD_GET,
43         extensions="txt")
44 @ServiceDescription("Simple Demo Servlet")
45 public class SimpleServlet extends SlingSafeMethodsServlet {
46
47     private static final long serialVersionUID = 1L;
48
49     @Override
50     protected void doGet(final SlingHttpServletRequest req,
51             final SlingHttpServletResponse resp) throws ServletException, IOException {
52         final Resource resource = req.getResource();
53         resp.setContentType("text/plain");
54         resp.getWriter().write("Title = " + resource.getValueMap().get(JcrConstants.JCR_TITLE));
55     }
56 }
57
```

For better understanding of the annotations please go through the sling documentation

https://sling.apache.org/documentation/the-sling-engine/servlets.html

Servlet Registration

A **SlingServletResolver** listens for Servlet services and - given the correct service registration properties - provides the servlets as resources in the (virtual) resource tree. Such servlets are provided as **ServletResource** instances which adapt to the **javax.servlet.Servlet** class.

For a Servlet registered as an OSGi service to be used by the Sling Servlet Resolver, either one or both of the **sling.servlet.paths** or the **sling.servlet.resourceTypes** service reference properties must be set. If neither is set, the Servlet service is ignored.

A Sling servlet can be registered in two ways -

1. **Using Resource Types** - Using this way, we use the **sling:resourceType** property of the node. For this, we need to hit the path in the browser for which the *sling:resourceType* is the given one.
2. **Using Paths** - Using this way, we can directly use the **path** specified in the request and our servlet will be executed.

Note: Registering a servlet via a path is relatively easier than registering it via a resource type but it is still recommended to use resource types instead of paths. Below are the reasons why -

- **If we use paths, then we need to be extra careful as we do not want to give any path randomly. The servlets cannot be access-controlled using the default JCR ACLs.**
- **No suffix handling in the path bound servlets.**
- **We also need to specify the paths to the servlet consumers and change in the path can have a serious impact.**

Types of Servlets

There are two types of servlets in Sling which are nothing but the classes we need to extend while creating our servlet.

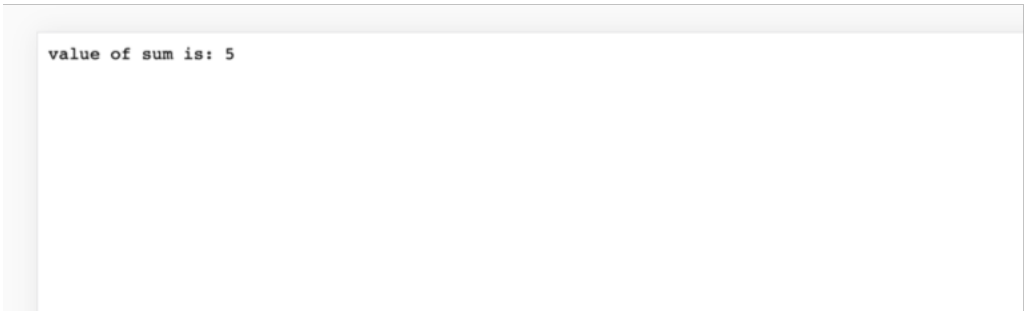
1. **SlingSafeMethodsServlet** - If we want to use only the read-only methods then we use this. This base class is actually just a better implementation of the Servlet API HttpServlet class which accounts for extensibility. So extensions of this class have great control over what methods to overwrite. It supports GET, HEAD, OPTIONS etc methods.
2. **SlingAllMethodsServlet** - If we want to use methods that write as well, then we use this. This class extends the SlingSafeMethodsServlet by support for the POST, PUT and DELETE methods.

Exercise 3: Create a resourceType based get servlet

This servlet is a get servlet. We can create a get or post servlet using any of the approach (path based or resourceType based). To create your servlet, follow the below steps:

1. Create a new servlet GetMathematicsOperationsServlet.java inside servlet package. Code is given inside the exercise folder.
2. Once your code is ready build your code and deploy it.
3. You should be able to access this servlet via this URL

   http://localhost:4502/content/mysite/language-masters/en/_jcr_content.mathsoperations.json
4. You should see a response like this on your page

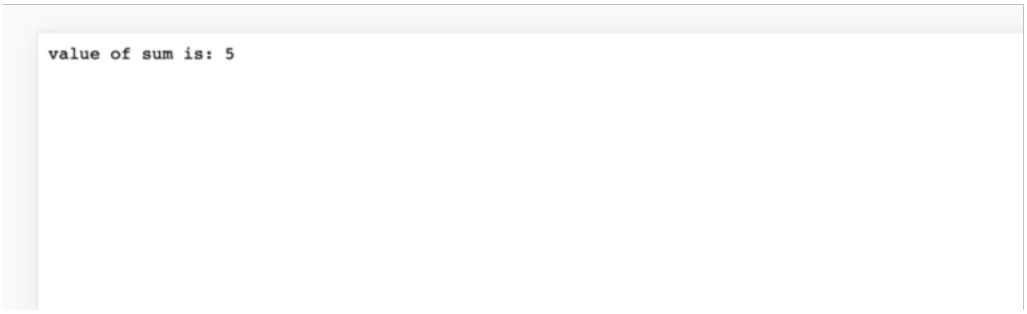


Exercise 4: Create a path based get servlet

To create a path based servlet, follow the below steps:

1. Create a new servlet PathBasedServlet.java inside servlet package. Code is given inside the exercise folder.
2. Once your code is ready build your code and deploy it.
3. You should be able to access this servlet via this URL

   http://localhost:4502/bin/pathbasedexample



4. You should see a response like this on your page

Exercise 5: Create a post servlet :

1. Create a new servlet PostServlet.java inside servlet package. Code is given inside the exercise folder.
2. Once your code is ready build your code and deploy it.
3. You should be able to access this via this ajax request from browser.

```
var formdata = {"firstNumber":'7',"secondNumber":"2"};
$.ajax({
type: "POST", url: "/bin/postservletexample",
data: { load: JSON.stringify(formdata) },
cache: false,
success: function(data){
alert("Success: "+JSON.stringify(data));
},
error: function(data){
alert("Error: "+JSON.stringify(data));
}
});
```